DBMS Zihin Haritası – Tekrar Notu

Veritabanı Yönetim Sistemleri (DBMS) için kapsamlı konu özeti. Tekrar yaparken bölümleri sırayla gözden geçir.

1. Temel Kavramlar: Veri ve Bilgi

- **Veri (Data):** Çevreden yapılan ölçüm ve gözlemlerle elde edilen, henüz işlenmemiş ham gerçeklerdir (sayı, harf, karakter, görüntü, ses vb.). İyi kararlar almak için gereken bilginin hammaddesidir.
- **Bilgi (Information):** Verinin anlamlı biçimde organize edilmesi/işlenmesiyle elde edilen sonuçtur.
- **Veritabanı (Database):** Belirli bir konuyla ilgili, düzenli saklanan, verimli depolama ve hızlı erişim için tasarlanmış yapılandırılmış veriler topluluğudur. Son kullanıcı verilerini ve metadata'yı saklayan paylaşımlı ve entegre bir yapıdır.
- **Metadata (Veri Hakkındaki Veri):** Veritabanındaki verilerin karakteristiklerini, veri tiplerini ve aralarındaki ilişkileri tanımlayan veridir.

2. Geleneksel Dosyalama Sistemleri ve Erişim Yöntemleri

Geçmişte bilgiler, birbirinden bağımsız bilgisayarlı dosyalarda saklanıyordu (File-Based Systems). Temel erişim yöntemleri:

- **Sequential (Sıralı) Erişim:** Dosya baştan sona okunur (eski müzik kasetleri gibi). Tüm kayıtlar sırayla işlenecekse avantajlıdır; belirli bir bilgiye doğrudan erişim gerekiyorsa büyük dezavantajdır.
- **Direct Access (Doğrudan Erişim):** Her satır için benzersiz, genellikle ardışık bir indeks numarası üretilir ve ayrı bir dosyada tutulur. İndeks değeri ile karşılık gelen disk adresi sayesinde verinin konumu doğrudan belirlenir ve oraya atlanır.

3. Dosya Tabanlı Sistemlerin Sınırlamaları

Terim (İngilizce)	Türkçe Karşılığı	Açıklama
Separation and isolation of data	Verilerin ayrılması ve yalıtılması	Veriler birbirine bağlantısız, izole dosyalarda bulunur. Her departmanın kendi dosyasını oluşturması yalıtıma yol açar; veritabanı uzmanları bu yapılar için "bilgi adaları (islands of information)" terimini kullanır.
Data dependence	Veri bağımlılığı	Bir dosyanın depolama özelliği değişirse (örn. tam sayı → ondalık) o dosyaya erişen tüm programların da değiştirilmesi gerekir. Sistemi hantal ve yönetilmesi zor yapar.
Incompatible file formats	Uyumsuz dosya formatları	Her dosya kendi uygulama programını kullanır ve onu oluşturan departmana aittir. Birbirine uymayan formatlar veri geçişini zorlaştırır.
Fixed queries / proliferation of application programs	Sabit sorgular / uygulama programlarının çoğalması	Anlık (ad hoc) sorgu yapmak imkânsızdır; en basit rapor için bile yeni program yazılır. Her yeni rapor talebinde program sayısı hızla artar (proliferation).
Data redundancy	Veri tekrarı	Aynı verinin gereksiz yere birden fazla yerde saklanmasıdır. Veri tutarsızlığına (data inconsistency) ve anormalliklere yol açar.

4. Veritabanı Yönetim Sistemi (DBMS) ve Faydaları

DBMS nedir? Kullanıcılar ile veritabanı arasında aracı (intermediary) görevi gören, veritabanı yapısını yöneten ve verilere erişimi kontrol eden yazılımlar bütünüdür. Örnekler: MS Access, Oracle, MySQL, MS SQL Server.

Temel faydaları:

- **Redundancy önlenir:** Veri tek yerde saklanır.
- **Consistency (tutarlılık):** Değişiklikler her yere yansır.
- **Sharing (paylaşım):** Aynı anda birden çok istemci erişebilir.
- **Integrity (bütünlük):** Kısıtlamalara uyum denetlenir.
- **Security (güvenlik):** Yetkisiz erişim engellenir.
- **Independence (bağımsızlık):** Fiziksel değişimler programları etkilemez.

5. Kritik Veri Özellikleri

- **Data Consistency (Veri Tutarlılığı):** Birden fazla kaynakta saklanan verinin doğruluğu ve tekdüzeliği. Bir işlem bittiğinde o veriye yapılan sonraki tüm erişimler güncel değeri yansıtmalıdır. Örnek: Adres değiştiğinde hem fatura hem kargo sistemine aynı anda yansımalı.
- **Data Integrity (Veri Bütünlüğü):** Verinin tüm yaşam döngüsünde doğru, eksiksiz ve gerçek dünya koşullarıyla uyumlu olmasının garanti altına alınması. Korumak için: veri girişi doğrulanmalı, kopyalar temizlenmeli, erişim kontrolleri sağlanmalı.
- **Data Concurrency (Veri Eşzamanlılığı):** Birden fazla kullanıcının/işlemin, sistem bütünlüğünü bozmadan verilere aynı anda (simultaneously) erişebilmesini sağlayan kontrol mekanizması.

6. Veri Soyutlama Dereceleri ve Mimariler

ANSI/SPARC Üç Seviyeli Mimari

1. **Dış Düzey / Dış Şema (External Level):** Son kullanıcının veya uygulamaların veritabanını gördüğü en üst düzey. Örn: İK sadece maaşı görür, şube sadece şube no'yu görür.
2. **Kavramsal Düzey / Kavramsal Şema (Conceptual Level):** Tüm veritabanının bütüncül, mantıksal organizasyonu. DBA tarafından yönetilir; tüm dış görünümlerin birleşimidir.
3. **İç Düzey / İç Şema (Internal Level):** Verinin diskte fiziksel olarak nasıl saklandığını ve kodlandığını gösteren teknik katman (struct blokları vb.).

Veri Bağımsızlığı (Data Independence)

- **Mantıksal Veri Bağımsızlığı:** Kavramsal şemadaki değişimin (tablo ekleme vb.) dış şemaları/programları etkilememesi.
- **Fiziksel Veri Bağımsızlığı:** İç düzeydeki depolama değişiminin kavramsal şemayı etkilememesi.

İstemci/Sunucu (Client/Server) Mimarisi

Kaynağın kullanıcısı (client) ile sağlayıcısı (server) ağ üzerinde ayrılır.

- **İstemci:** Arayüzü ve sorgu isteğini yönetir.
- **Sunucu:** Yetkilendirme, bütünlük kontrolü ve veritabanı işlemlerini yürütür.

Bulut (Cloud) Hizmet Modelleri

Model	Sağlayıcıda olanlar	Sizde olanlar
IaaS (Altyapı)	Donanım/sunucu	İşletim sistemi ve uygulama
PaaS (Platform)	İşletim sistemi ve altyapı	Uygulama ve veri
SaaS (Yazılım)	Her şey	Sadece hizmeti kullanırsınız (örn. Gmail, Netflix – çoklu kiracı / multi-tenancy)

7. Sorgu İşleme (Query Processing) Adımları

Bir SQL sorgusu çalıştırıldığında sırasıyla:

1. **Parser (Ayrıştırıcı):** SQL kelimelerinin (Select, From, Where vb.) ve sözdiziminin doğru sırada yazılıp yazılmadığını kontrol eder; hata varsa uyarır.
2. **Translator (Çevirici):** Doğrulanan SQL ifadesini ilişkisel cebir denklemlerine dönüştürür.
3. **Optimizer (En İyileyici):** Veritabanı istatistiklerini kullanarak sorgunun en hızlı ve en düşük maliyetle çalışacağı çalışma planını (execution plan) hazırlar.
4. **Evaluation Engine (Değerlendirme Motoru):** Planı çalıştırarak verileri işler ve sonuç kümesini (query output) üretir.

8. Veri Modelleri ve İlişkisel Model Temelleri

Geçmişten Günümüze Modeller (Kısa Özet)

- **Hiyerarşik Model:** 1960–70'ler, ağaç yapısı, 1 ebeveyn – çok çocuk kuralı; veri tekrarı dezavantajı.
- **Ağ Modeli:** Hiyerarşik kısıtları aşar, çoktan-çoğa ilişkiyi destekler; ancak işaretçiler (pointers) yüzünden çok karmaşıktır.
- **İlişkisel Model:** E.F. Codd (1980'ler), satır ve sütunlardan oluşan tablolar, PK/FK yapısı. Hâlâ en yaygın modeldir.
- **Nesne Yönelimli Model:** 1990'lar, sınıf (class) ve kalıtım (inheritance) mantığı; karmaşık verilerde yüksek performans.
- **Yeni Modeller:** Nesne/İlişkisel hibrit model; platformlar arası veri değişimi için XML ve JSON.

8.1 Hierarchical Data Model (Hiyerarşik Veri Modeli)

- 1960'lar ve 1970'lerde kullanılmıştır; kayıtlar (records) bir **ağaç (tree)** yapısında saklanır.
- Her dalın (branch) alt dalları sayesinde dallanma ve çeşitlilik artar.
- Temeli **ebeveyn-çocuk (parent-child)** ilişkisine dayanır: bir ebeveynin birden çok çocuğu olabilir, bir çocuğun yalnızca bir ebeveyni olur.
- Bir alt dal, ebeveyn dalına yalnızca tek bir noktadan bağlanmak zorundadır. Bir düğüm (node) alt seviyedeki *n* adet düğüme, üst seviyedeki ise sadece 1 düğüme bağlanabilir.
- Varlıklar (entities) arasında ilişki kurarken her varlık yalnızca bir diğer varlığa bağlanmalıdır.
- **Dezavantaj – veri tekrarı:** Aynı varlığın birden fazla ilişkide kullanılması gerekiyorsa varlığın yeniden oluşturulması gerekir; bu da gereksiz veri tekrarına (data duplication / redundancy) yol açar. *Örnek:* "Departman 1" hem "Proje Yöneticisi 1" altında hem de başka bir yönetici altında gerekiyorsa, tek ebeveyne bağlanabildiği için bu departman kaydı kopyalanarak yeniden oluşturulmalıdır.
- Herhangi bir bilgi aranırken arama işlemi en tepeden başlar.
- Bir dal silindiğinde, o dala bağlı tüm kayıtlar da otomatik olarak silinir.

8.2 Network Data Model (Ağ Veri Modeli)

- **Neden çıktı:** Hiyerarşik modeldeki kısıtlamaları (bir düğümün üst seviyede sadece 1 düğüme bağlanabilmesi) aşmak, karmaşık ilişkileri (çoktan-çoğa / many-to-many) daha etkili temsil etmek ve veritabanı performansını artırmak için geliştirilmiştir.
- **Neyi önledi:** Hiyerarşik modeldeki bağlantı kısıtlamasını kaldırarak bir elemanın herhangi bir diğer elemana bağlanabilmesine (her kaydın birden fazla ebeveyni ve çocuğu olabilmesine) olanak tanımıştır.
- **Dezavantaj – karmaşıklık:** Düğümler arasındaki bağlantılar işaretçiler (pointers) ile kurulduğundan yapı büyüdükçe işaretçi kullanımı uygulamalarda çok karmaşık durumlara yol açar ve yönetimi zorlaştırır.
- **Dezavantaj – kullanıcı dostu olmaması:** Tasarım ve yapı user-friendly değildir; sistem büyüdükçe verinin takibi giderek zorlaşır.
- Anlık sorgu (ad hoc query) yeteneğinin eksikliği ve veritabanı yapısındaki bir değişikliğin tüm programlarda sorun yaratması (**yapısal bağımlılık**) nedeniyle 1980'lerde yerini büyük ölçüde İlişkisel (Relational) modele bırakmıştır.

8.3 Relational Data Model (İlişkisel Veri Modeli)

- **Neden çıktı:** Hiyerarşik ve ağ modellerinin dezavantajlarını gidermek için E.F. Codd tarafından 1980'lerde geliştirilmiştir.
- **Avantajları:** Daha basit veri temsili, üstün veri bağımsızlığı (structural independence) ve kullanımı kolay sorgu dili (SQL ile anlık / ad hoc sorgular).
- Ortak özelliklere sahip veriler satırlardan (rows) ve sütunlardan (columns) oluşan tablolarda (tables) saklanır.
- Veriler ve ilişkiler tamamen tablolar üzerinde tanımlanır; tüm bilgiler şeffaf bir şekilde görünür.
- Tablolar arası ilişkiler **Primary Key (PK)** ve **Foreign Key (FK)** ile kurulur.
- **Dezavantajları:** Kavramsal sadeliği, eğitimsiz kişilerin sistemi kötü kullanmasına zemin hazırlayabilir; kontrol edilmezse dosya sistemlerindeki gibi veri anormalliklerine ve "bilgi adaları (islands of information)" sorunlarına yol açabilir.

8.4 Object-Oriented Data Model (Nesne Yönelimli Veri Modeli)

- **Neden çıktı:** Zengin bir anlamsal (semantic) çerçeve içinde karmaşık nesneleri (complex objects) ve verileri desteklemek için 1990'larda ortaya çıkmıştır.
- İlişkisel modeldeki **ilişki (relation)** kavramı, bu modelde **sınıf (class)** kavramına karşılık gelir.
- Veriler nesneler (objects) olarak modellenir ve oluşturulur; OOP'deki sınıf (class) ve kalıtım / miras alma (inheritance) kavramları doğrudan geçerlidir.
- **Avantajları:** Yapısal olarak çok karmaşık verileri işlerken yüksek performans sunar; veri tipleri esnektir, veri yapısında genişleme ve yeniden düzenleme için büyük esneklik sağlar; özellikle büyük tablolarda arama (search) işlemleri önemli ölçüde daha hızlıdır.

8.5 New Data Models (Yeni Veri Modelleri)

- **Neden çıktı:** Önceki modellerin yetersiz kaldığı yapılandırılmamış (unstructured) verileri işlemek ve platformlar arası veri değişimini kolaylaştırmak için ortaya çıkmışlardır.
- **Object/Relational Model:** İlişkisel model ile nesne yönelimli modelin birleştirilmesiyle ortaya çıkmış hibrit bir veri modelidir.
- **XML:** Çapraz platform veri değişimi (cross-platform data exchange) için kullanılan bir veri tanımlama standardıdır; yapılandırılmamış verileri tanımlamak için de kullanılır.
- **JSON:** XML gibi, öncelikli olarak platformlar arası veri değişimi için kullanılan bir standarttır ve yapılandırılmamış verileri tanımlamak için kullanılır.

8.6 İlişkisel Model Temel Kavramları (Basic Database Concepts)

- **Tablo / İlişki (Table / Relation), Sütun / Alan (Column / Field), Satır / Kayıt (Row / Record):** Veriler satır/sütun matrisleri halinde organize edilir; alanlar tablo özniteliklerine (attribute) karşılık gelir.
- **Data Type (Veri Tipi):** Veritabanındaki verinin türünü belirtir.
- **Null:** İçinde hiçbir değer barındırmayan boşluk durumudur.
- **Record / Row (Kayıt / Satır):** Bir tablo içindeki yatay veri dizilimidir.
- **Result Set (Sonuç Kümesi):** Sorgular sonucunda dönen veri kümesidir.

Constraints (Kısıtlamalar)

Veri bütünlüğünü koruyan kurallardır.

Kısıtlama	Görevi
Primary Key (PK)	Her satırı tamamen benzersiz yapar; boş olamaz. Tablolar arasında ilişki kurarken kullanılan temel tanımlayıcıdır.
Foreign Key (FK)	Başka bir tabloyla ilişki kurar ve geçerli kayıt referansını denetler.
Unique Key	Belirli sütunda tekrarı engeller (benzersizlik).
Default	Değer boş kalırsa atanacak varsayılan değeri belirler.
Check	Girilen verinin özel bir koşula uyup uymadığını denetler.

Indexes (İndeksler)

Hızlı erişim sağlar.

- **Clustered Index:** Verileri diskte fiziksel olarak sıralar (physical order).
- **Non-Clustered Index:** Fiziksel sırayı değiştirmez (non-physically ordered); ayrı bir mantıksal arama haritası sunar.

Hızlı Tekrar Soruları

- Veri ile bilgi arasındaki fark nedir?
- Dosya tabanlı sistemlerin 5 sınırlaması nelerdir?
- Consistency, integrity ve concurrency arasındaki fark nedir?
- ANSI/SPARC'ın 3 seviyesi ve iki tür veri bağımsızlığı nedir?
- IaaS, PaaS ve SaaS'ta kim neyi yönetir?
- Sorgu işleme 4 adımı sırasıyla nedir?
- Hiyerarşik modelde veri tekrarı neden oluşur? Ağ modeli bunu nasıl aşmıştır?
- Ağ modeli neden yerini ilişkisel modele bırakmıştır?
- PK, FK, Unique, Default ve Check farkları nedir?
- Clustered ve Non-Clustered index farkı nedir?